

A simple way to manage an instance is to make use of an SSH connection, for which Amazon provides the means for generating the appropriate keys.

The EC2 environment in which an instance is executed provides different levels of the following services:

- **CPU:** allows to select the number and type of cores, including GPUs
- **Memory:** defines how much main memory is allocated to an instance
- **Storage:** defines how much local storage is allocated
- **Platform:** distinguishes between 32-bit or 64-bit architectures
- **Networking:** sets the bandwidth capacity that can be used

In addition, extra resources can be requested such as an additional networking interface. The local storage that comes with an instance is *transient*: when the instance stops, all the data stored locally is lost. In order to prevent data loss, a customer will need to explicitly save data to persistent store, for example, by making use of Amazon's Simple Storage Service (S3). An alternative is to attach a storage device that is mapped to Amazon's **Elastic Block Store (Amazon EBS)**. Again, this is yet another service, but one that can be used in the form of a virtual block device that is simply mounted as one would mount an additional hard disk. When an instance is stopped, all data that was stored on EBS will persist. And just as one would expect, an EBS device can be (re)mounted to any other instance as well.

It should be clear by now that, without having gone into any significant level of detail, the IaaS as offered by EC2 allows a customer to create a (potentially large) number of virtual machines, each configured with resources as needed, and capable of exchanging messages through an IP network. In addition, these virtual machines can be accessed from anywhere over the Internet (provided a client has the proper credentials). As such, Amazon EC2, like many other IaaS providers, offers the means to configure a complete distributed system consisting of networked virtual servers and running customer-supplied distributed applications. At the same time, those customers will not need to maintain any physical machine, which by itself is often already a huge gain as we will encounter at several occasions throughout this text. One can indeed argue that virtualization lies at the core of modern cloud computing.

3.3 Clients

In the previous chapters we discussed the client-server model, the roles of clients and servers, and the ways they interact. Let us now take a closer look at the anatomy of clients and servers, respectively. We start in this section with a discussion of clients. Servers are discussed in the next section.

Networked user interfaces

A major task of client machines is to provide the means for users to interact with remote servers. There are roughly two ways in which this interaction can be supported. First, for each remote service the client machine will have a separate counterpart that can contact the service over the network. A typical example is a calendar running on a user's smartphone that needs to synchronize with a remote, possibly shared calendar. In this case, an application-level protocol will handle the synchronization, as shown in Figure 3.10(a).

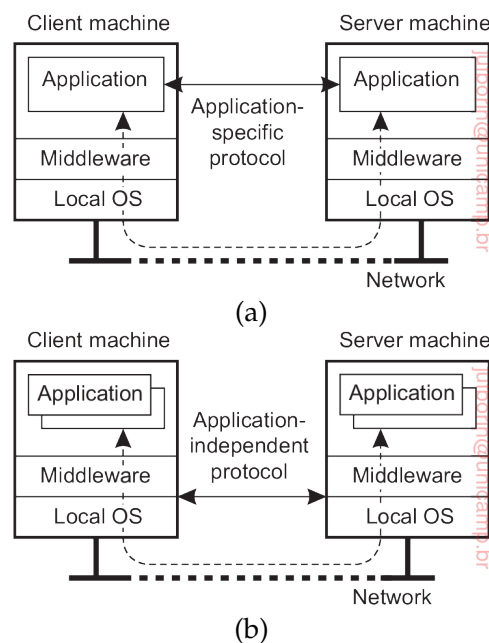


Figure 3.10: (a) A networked application with its own protocol. (b) A general solution to allow access to remote applications.

A second solution is to provide direct access to remote services by offering only a convenient user interface. Effectively, this means that the client machine is used only as a terminal with no need for local storage, leading to an application-neutral solution as shown in Figure 3.10(b). In the case of networked user interfaces, everything is processed and stored at the server. This **thin-client approach** has received much attention with the increase of Internet connectivity and the use of mobile devices. Thin-client solutions are also popular as they ease the task of system management.

Example: The X window system

Perhaps one of the oldest and still widely used networked user interfaces is the **X Window System**. The X Window System, generally referred to simply as **X**, is used to control bit-mapped terminals, which include a monitor, keyboard, and a pointing device such as a mouse. Next to supporting traditional terminals as can be found with desktop computers and workstations, X also supports modern devices such as touchscreens on tablets and smartphones. In a sense, X can be viewed as that part of an operating system that controls the terminal. The heart of the system is formed by what we shall call the **X kernel**. It contains all the terminal-specific device drivers, and as such, is generally highly hardware dependent.

The X kernel offers a relatively low-level interface for controlling the screen, but also for capturing events from the keyboard and mouse. This interface is made available to applications as a library called Xlib. This general organization is shown in Figure 3.11. Note that Xlib is hardly ever used directly by applications, which instead deploy easier to use toolkits implemented on top of Xlib.

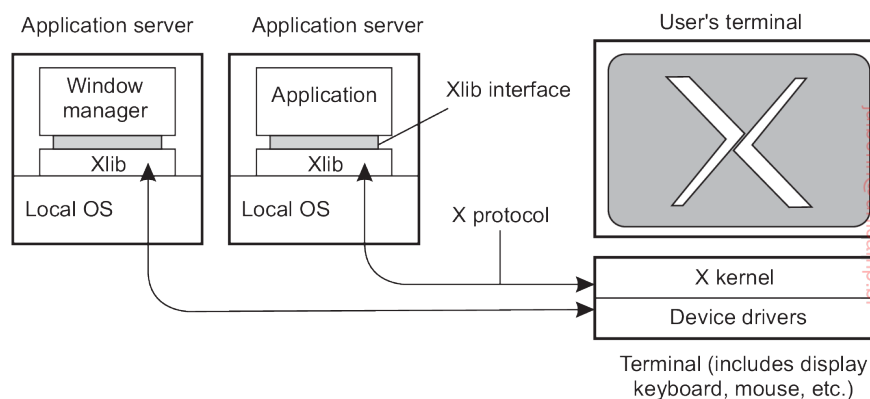


Figure 3.11: The basic organization of the X Window System.

The interesting aspect of X is that the X kernel and the X applications need not necessarily reside on the same machine. In particular, X provides the **X protocol**, which is an application-level communication protocol by which an instance of Xlib can exchange data and events with an X kernel. For example, Xlib can send requests to the X kernel for creating or killing a window, setting colors, and defining the type of cursor to display, among many other requests. In turn, the X kernel will react to local events such as keyboard and mouse input by sending event packets back to Xlib.

Several applications can communicate at the same time with the X kernel. There is one specific application that is given special rights, known as the **window manager**. This application can dictate the “look and feel” of the

display as it appears to the user. For example, the window manager can prescribe how each window is decorated with extra buttons, how windows are to be placed on the display, and so on. Other applications will have to adhere to these rules. In practice, this means that much of the interaction between an application and an X terminal is redirected through a window manager.

It is interesting to note how the X window system actually fits into client-server computing. From what we have described so far, it should be clear that the X kernel receives requests to manipulate the display. It gets these requests from (possibly remote) applications. In this sense, the X kernel acts as a server, while the applications play the role of clients. This terminology has been adopted by X, and although strictly speaking it is correct, it can easily lead to confusion.

Thin-client network computing

Obviously, applications manipulate a display using the specific display commands as offered by X. These commands are generally sent over the network where they are subsequently executed by the X kernel. By its nature, applications written for X should preferably separate application logic from user-interface commands. Unfortunately, this is often not the case. As reported by Lai and Nieh [2002] it turns out that much of the application logic and user interaction are tightly coupled, meaning that an application will send many requests to the X kernel for which it will expect a response before being able to make a next step. This synchronous behavior may adversely affect performance when operating over a wide-area network with long latencies.

There are several solutions to this problem. One is to re-engineer the implementation of the X protocol, as is done with NX [Pinzari, 2003]. An important part of this work concentrates on bandwidth reduction by reducing the size of X messages. To this end, messages are considered to consist of a fixed part, which is treated as an identifier, and a variable part. In many cases, multiple messages will have the same identifier in which case they will often contain similar data. This property can be used to send only the differences between messages having the same identifier. By having the sender and receiver maintain identifiers, decoding at the receiver can be readily applied. Bandwidth reductions up to a factor 1000 have been reported, which allows X to also run through low-bandwidth links of only 9600 kbps.

As an alternative to using X, researchers and practitioners have also sought to let an application *completely* control the remote display, that is, up the pixel level. Changes in the bitmap are then sent over the network to the display, where they are immediately transferred to the local frame buffer. A well-known example of this approach is **Virtual Network Computing (VNC)** [Richardson et al., 1998], which has been around ever since the late 1990s. Obviously, letting the application control the display requires sophis-

ticated encoding techniques in order to prevent bandwidth availability to become a problem. For example, consider displaying a video stream at a rate of 30 frames per second on a simple 320×240 screen. If each pixel is encoded by 24 bits, then without an efficient encoding scheme, we would need a bandwidth of approximately 53 Mbps. In practice, various encoding techniques are used, yet choosing the best one is generally application dependent.

The drawback of sending raw pixel data in comparison to higher-level protocols such as X is that it is impossible to make any use of application semantics, as these are effectively lost at that level. Baratto et al. [2005] propose a different technique. In their solution, referred to as THINC, they provide a few high-level display commands that operate at the level of the video device drivers. These commands are thus device dependent, more powerful than raw pixel operations, but less powerful compared to what a protocol such as X offers. The result is that display servers can be much simpler, which is good for CPU usage, while at the same time application-dependent optimizations can be used to reduce bandwidth and synchronization.

Client-side software for distribution transparency

Client software comprises more than just user interfaces. In many cases, parts of the processing and data level in a client-server application are executed on the client side as well. A special class is formed by embedded client software, such as for automatic teller machines (ATMs), cash registers, barcode readers, TV set-top boxes, etc. In these cases, the user interface is a relatively small part of the client software, in contrast to the local processing and communication facilities.

Besides the user interface and other application-related software, client software comprises components for achieving **distribution transparency**. Ideally, a client should not be aware that it is communicating with remote processes. In contrast, distribution is often less transparent to servers for reasons of performance and correctness.

Access transparency is generally handled through the generation of a **client stub** from an interface definition of what the server has to offer. The stub provides the same interface as the one available at the server, but hides the possible differences in machine architectures, as well as the actual communication. The client stub transforms local calls to messages that are sent to the server, and *vice versa* transforms messages from the server to return values as one would expect when calling an ordinary procedure.

There are different ways to handle **location, migration, and relocation transparency**. Using a convenient naming system is crucial. In many cases, cooperation with client-side software is also important. For example, when a client is already bound to a server, the client can be directly informed when the server changes location. In this case, the client's middleware can hide the server's current network location from the user, and also transparently

rebind to the server if necessary. At worst, the client's application may notice a temporary loss of performance.

In a similar way, many distributed systems implement **replication transparency** by means of client-side solutions. For example, imagine a distributed system with replicated servers. Such replication can be achieved by forwarding a request to each replica, as shown in Figure 3.12. Client-side software can transparently collect all responses and pass a single response to the client application.

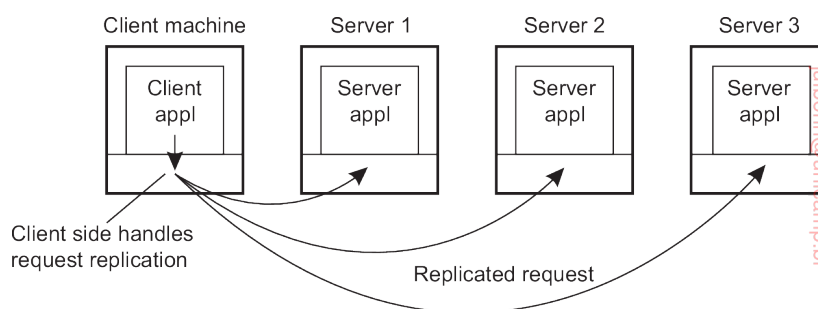


Figure 3.12: Transparent replication of a server using a client-side solution.

Regarding **failure transparency**, masking communication failures with a server is typically done through client middleware. For example, client middleware can be configured to repeatedly attempt to connect to a server, or perhaps try another server after several attempts. There are even situations in which the client middleware returns data it had cached during a previous session, as is sometimes done by Web browsers that fail to connect to a server.

Finally, **concurrency transparency** can be handled through special intermediate servers, notably transaction monitors, and requires less support from client software.

3.4 Servers

Let us now take a closer look at the organization of servers. In the following pages, we first concentrate on a number of general design issues for servers, followed by a discussion on server clusters.

General design issues

A server is a process implementing a specific service on behalf of a collection of clients. In essence, each server is organized in the same way: it waits for an incoming request from a client and subsequently ensures that the request is taken care of, after which it waits for the next incoming request.

Concurrent versus iterative servers

There are several ways to organize servers. In the case of an **iterative server**, the server itself handles the request and, if necessary, returns a response to the requesting client. A **concurrent server** does not handle the request itself, but passes it to a separate thread or another process, after which it immediately waits for the next incoming request. A multithreaded server is an example of a concurrent server. An alternative implementation of a concurrent server is to fork a new process for each new incoming request. This approach is followed in many Unix systems. The thread or process that handles the request is responsible for returning a response to the requesting client.

Contacting a server: end points

Another issue is where clients contact a server. In all cases, clients send requests to an **end point**, also called a **port**, at the machine where the server is running. Each server listens to a specific end point. How do clients know the end point of a service? One approach is to globally assign end points for well-known services. For example, servers that handle Internet FTP requests always listen to TCP port 21. Likewise, an HTTP server for the World Wide Web will always listen to TCP port 80. These end points have been assigned by the Internet Assigned Numbers Authority (IANA), and are documented in [Reynolds and Postel, 1994]. With assigned end points, the client needs to find only the network address of the machine where the server is running. Name services can be used for that purpose.

There are many services that do not require a preassigned end point. For example, a time-of-day server may use an end point that is dynamically assigned to it by its local operating system. In that case, a client will first have to look up the end point. One solution is to have a special daemon running on each machine that runs servers. The daemon keeps track of the current end point of each service implemented by a co-located server. The daemon itself listens to a well-known end point. A client will first contact the daemon, request the end point, and then contact the specific server, as shown in Figure 3.13(a).

It is common to associate an end point with a specific service. However, actually implementing each service by means of a separate server may be a waste of resources. For example, in a typical Unix system, it is common to have lots of servers running simultaneously, with most of them passively waiting until a client request comes in. Instead of having to keep track of so many passive processes, it is often more efficient to have a single **superserver** listening to each end point associated with a specific service, as shown in Figure 3.13(b). For example, the `inetd` daemon in Unix listens to a number of well-known ports for Internet services. When a request comes in, the daemon forks a process to handle it. That process will exit when finished.

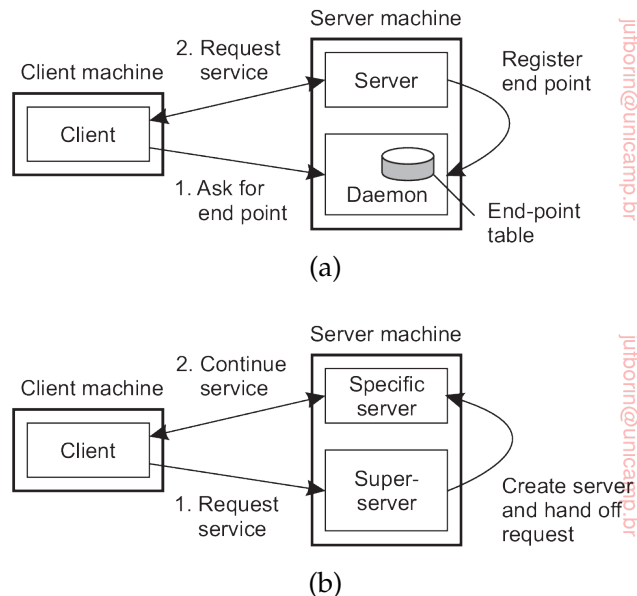


Figure 3.13: (a) Client-to-server binding using a daemon. (b) Client-to-server binding using a superserver.

Interrupting a server

Another issue that needs to be taken into account when designing a server is whether and how a server can be interrupted. For example, consider a user who has just decided to upload a huge file to an FTP server. Then, suddenly realizing that it is the wrong file, he wants to interrupt the server to cancel further data transmission. There are several ways to do this. One approach that works only too well in the current Internet (and is sometimes the only alternative) is for the user to abruptly exit the client application (which will automatically break the connection to the server), immediately restart it, and pretend nothing happened. The server will eventually tear down the old connection, thinking the client has probably crashed.

A much better approach for handling communication interrupts is to develop the client and server such that it is possible to send **out-of-band data**, which is data that is to be processed by the server before any other data from that client. One solution is to let the server listen to a separate control end point to which the client sends out-of-band data, while at the same time listening (with a lower priority) to the end point through which the normal data passes. Another solution is to send out-of-band data across the same connection through which the client is sending the original request. In TCP, for example, it is possible to transmit urgent data. When urgent data are received at the server, the latter is interrupted (e.g., through a signal in Unix

systems), after which it can inspect the data and handle them accordingly.

Stateless versus stateful servers

A final, important design issue, is whether or not the server is stateless. A **stateless server** does not keep information on the state of its clients, and can change its own state without having to inform any client [Birman, 2012]. A Web server, for example, is stateless. It merely responds to incoming HTTP requests, which can be either for uploading a file to the server or (most often) for fetching a file. When the request has been processed, the Web server forgets the client completely. Likewise, the collection of files that a Web server manages (possibly in cooperation with a file server), can be changed without clients having to be informed.

Note that in many stateless designs, the server actually does maintain information on its clients, but crucial is the fact that if this information is lost, it will not lead to a disruption of the service offered by the server. For example, a Web server generally logs all client requests. This information is useful, for example, to decide whether certain documents should be replicated, and where they should be replicated to. Clearly, there is no penalty other than perhaps in the form of suboptimal performance if the log is lost.

A particular form of a stateless design is where the server maintains what is known as **soft state**. In this case, the server promises to maintain state on behalf of the client, but only for a limited time. After that time has expired, the server falls back to default behavior, thereby discarding any information it kept on account of the associated client. An example of this type of state is a server promising to keep a client informed about updates, but only for a limited time. After that, the client is required to poll the server for updates. Soft-state approaches originate from protocol design in computer networks, but can be equally applied to server design [Clark, 1989; Lui et al., 2004].

In contrast, a **stateful server** generally maintains persistent information on its clients. This means that the information needs to be explicitly deleted by the server. A typical example is a file server that allows a client to keep a local copy of a file, even for performing update operations. Such a server would maintain a table containing *(client, file)* entries. Such a table allows the server to keep track of which client currently has the update permissions on which file, and thus possibly also the most recent version of that file.

This approach can improve the performance of read and write operations as perceived by the client. Performance improvement over stateless servers is often an important benefit of stateful designs. However, the example also illustrates the major drawback of stateful servers. If the server crashes, it has to recover its table of *(client, file)* entries, or otherwise it cannot guarantee that it has processed the most recent updates on a file. In general, a stateful server needs to recover its entire state as it was just before the crash. Enabling recovery can introduce considerable complexity, as we discuss in Chapter 8.

In a stateless design, no special measures need to be taken at all for a crashed server to recover. It simply starts running again, and waits for client requests to come in.

Ling et al. [2004] argue that one should actually make a distinction between (temporary) **session state** and **permanent state**. The example above is typical for session state: it is associated with a series of operations by a single user and should be maintained for a some time, but not indefinitely. As it turns out, session state is often maintained in three-tiered client-server architectures, where the application server actually needs to access a database server through a series of queries before being able to respond to the requesting client. The issue here is that no real harm is done if session state is lost, provided that the client can simply re-issue the original request. This observation allows for simpler and less reliable storage of state.

What remains for permanent state is typically information maintained in databases, such as customer information, keys associated with purchased software, etc. However, for most distributed systems, maintaining session state already implies a stateful design requiring special measures when failures do happen and making explicit assumptions about the durability of state stored at the server. We will return to these matters extensively when discussing fault tolerance.

When designing a server, the choice for a stateless or stateful design should not affect the services provided by the server. For example, if files have to be opened before they can be read from, or written to, then a stateless server should one way or the other mimic this behavior. A common solution is that the server responds to a read or write request by first opening the referred file, then does the actual read or write operation, and immediately closes the file again.

In other cases, a server may want to keep a record on a client's behavior so that it can more effectively respond to its requests. For example, Web servers sometimes offer the possibility to immediately direct a client to his favorite pages. This approach is possible only if the server has history information on that client. When the server cannot maintain state, a common solution is then to let the client send along additional information on its previous accesses. In the case of the Web, this information is often transparently stored by the client's browser in what is called a **cookie**, which is a small piece of data containing client-specific information that is of interest to the server. Cookies are never executed by a browser; they are merely stored.

The first time a client accesses a server, the latter sends a cookie along with the requested Web pages back to the browser, after which the browser safely tucks the cookie away. Each subsequent time the client accesses the server, its cookie for that server is sent along with the request.

Object servers

Let us take a look at the general organization of object servers needed for distributed objects. The important difference between a general object server and other (more traditional) servers is that an object server by itself does not provide a specific service. Specific services are implemented by the objects that reside in the server. Essentially, the server provides only the means to invoke local objects, based on requests from remote clients. As a consequence, it is relatively easy to change services by simply adding and removing objects.

An object server thus acts as a place where objects live. An object consists of two parts: data representing its state and the code for executing its methods. Whether or not these parts are separated, or whether method implementations are shared by multiple objects, depends on the object server. Also, there are differences in the way an object server invokes its objects. For example, in a multithreaded server, each object may be assigned a separate thread, or a separate thread may be used for each invocation request. These and other issues are discussed next.

For an object to be invoked, the object server needs to know which code to execute, on which data it should operate, whether it should start a separate thread to take care of the invocation, and so on. A simple approach is to assume that all objects look alike and that there is only one way to invoke an object. Unfortunately, such an approach is generally inflexible and often unnecessarily constrains developers of distributed objects.

A much better approach is for a server to support different policies. Consider, for example, a **transient object**: an object that exists only as long as its server exists, but possibly for a shorter period of time. An in-memory, read-only copy of a file could typically be implemented as a transient object. Likewise, a calculator could also be implemented as a transient object. A reasonable policy is to create a transient object at the first invocation request and to destroy it as soon as no clients are bound to it anymore.

The advantage of this approach is that a transient object will need a server's resources only as long as the object is really needed. The drawback is that an invocation may take some time to complete, because the object needs to be created first. Therefore, an alternative policy is sometimes to create all transient objects at the time the server is initialized, at the cost of consuming resources even when no client is making use of the object.

In a similar fashion, a server could follow the policy that each of its objects is placed in a memory segment of its own. In other words, objects share neither code nor data. Such a policy may be necessary when an object implementation does not separate code and data, or when objects need to be separated for security reasons. In the latter case, the server will need to provide special measures, or require support from the underlying operating system, to ensure that segment boundaries are not violated.

The alternative approach is to let objects at least share their code. For

example, a database containing objects that belong to the same class can be efficiently implemented by loading the class implementation only once into the server. When a request for an object invocation comes in, the server need only fetch that object's state and execute the requested method.

Likewise, there are many different policies with respect to threading. The simplest approach is to implement the server with only a single thread of control. Alternatively, the server may have several threads, one for each of its objects. Whenever an invocation request comes in for an object, the server passes the request to the thread responsible for that object. If the thread is currently busy, the request is temporarily queued.

The advantage of this approach is that objects are automatically protected against concurrent access: all invocations are serialized through the single thread associated with the object. Neat and simple. Of course, it is also possible to use a separate thread for each invocation request, requiring that objects should have already been protected against concurrent access. Independent of using a thread per object or thread per method is the choice of whether threads are created on demand or the server maintains a pool of threads. Generally there is no single best policy. Which one to use depends on whether threads are available, how much performance matters, and similar factors.

Decisions on how to invoke an object are commonly referred to as **activation policies**, to emphasize that in many cases the object itself must first be brought into the server's address space (i.e., activated) before it can actually be invoked. What is needed then is a mechanism to group objects per policy. Such a mechanism is sometimes called an **object adapter**, or alternatively an **object wrapper**. An object adapter can best be thought of as software implementing a specific activation policy. The main issue, however, is that object adapters come as generic components to assist developers of distributed objects, and which need only to be configured for a specific policy.

An object adapter has one or more objects under its control. Because a server should be capable of simultaneously supporting objects that require different activation policies, several object adapters may reside in the same server. When an invocation request is delivered to the server, the request is first dispatched to the appropriate object adapter, as shown in Figure 3.14.

An important observation is that object adapters are unaware of the specific interfaces of the objects they control. Otherwise, they could never be generic. The only issue that is important to an object adapter is that it can extract an object reference from an invocation request, and subsequently dispatch the request to the referenced object, but now following a specific activation policy. As is also illustrated in Figure 3.14 rather than passing the request directly to the object, an adapter hands an invocation request to the server-side stub of that object. The stub, also called a skeleton, is normally generated from the interface definitions of the object, unmarshals the request and invokes the appropriate method.

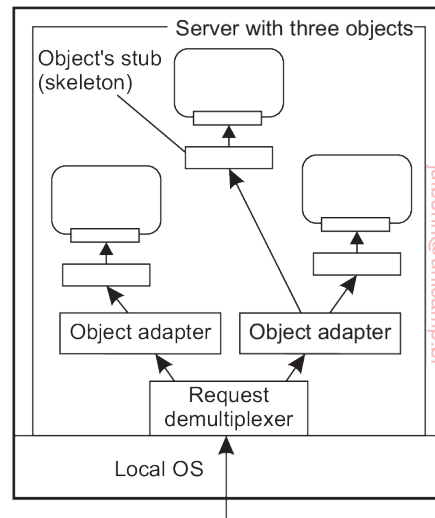


Figure 3.14: An object server supporting different activation policies.

An object adapter can support different activation policies by simply configuring it at runtime. For example, in CORBA-compliant systems [OMG, 2001], it is possible to specify whether an object should continue to exist after its associated adapter has stopped. Likewise, an adapter can be configured to generate object identifiers, or to let the application provide one. As a final example, an adapter can be configured to operate in single-threaded or multithreaded mode as we explained above.

Note that although in Figure 3.14 we have spoken about objects, we have said nothing about what these objects actually are. In particular, it should be stressed that as part of the implementation of such an object the server may (indirectly) access databases or call special library routines. The implementation details are hidden for the object adapter who communicates only with a skeleton. As such, the actual implementation may have nothing to do with what we often see with language-level (i.e., compile-time) objects. For this reason, a different terminology is generally adopted. A **servant** is the general term for a piece of code that forms the implementation of an object.

Note 3.6 (Example: The Ice runtime system)

Let us briefly consider the Ice distributed-object system, which has been partly developed in response to the intricacies of commercial object-based distributed systems [Henning, 2004]. An object server in Ice is nothing but an ordinary process that simply starts with initializing the Ice runtime system (RTS). The basis of the runtime environment is formed by what is called a *communicator*. A communicator is a component that manages a number of basic resources, of which the most important one is formed by a pool of threads. Likewise, it will have

associated dynamically allocated memory, and so on. In addition, a communicator provides the means for configuring the environment. For example, it is possible to specify maximum message lengths, maximum invocation retries, and so on.

Normally, an object server would have only a single communicator. However, when different applications need to be fully separated and protected from each other, a separate communicator (with possibly a different configuration) can be created within the same process. At the very least, such an approach would separate the different thread pools so that if one application has consumed all its threads, then this would not affect the other application.

A communicator can also be used to create an object adapter, such as shown in Figure 3.15. We note that the code is simplified and incomplete. More examples and detailed information on Ice can be found in Henning and Spruiell [2005].

```
main(int argc, char* argv[]) {
    Ice::Communicator ic;
    Ice::ObjectAdapter adapter;
    Ice::Object      object;

    ic = Ice::initialize(argc, argv);
    adapter = ic->createObjectAdapterWithEndpoints( "MyAdapter", "tcp -p 10000");
    object = new MyObject;
    adapter->add(object, objectID);
    adapter->activate();
    ic->waitForShutdown();
}
```

Figure 3.15: Example of creating an object server in Ice.

In this example, we start with creating and initializing the runtime environment. When that is done, an object adapter is created. In this case, it is instructed to listen for incoming TCP connections on port 10000. Note that the adapter is created in the context of the just created communicator. We are now in the position to create an object and to subsequently add that object to the adapter. Finally, the adapter is *activated*, meaning that, under the hood, a thread is activated that will start listening for incoming requests.

This code does not yet show much differentiation in activation policies. Policies can be changed by modifying the *properties* of an adapter. One family of properties is related to maintaining an adapter-specific set of threads that are used for handling incoming requests. For example, one can specify that there should always be only one thread, effectively serializing all accesses to objects that have been added to the adapter.

Again, note that we have not specified `MyObject`. Like before, this could be a simple C++ object, but also one that accesses databases and other external services that jointly implement an object. By registering `MyObject` with an adapter, such implementation details are completely hidden from clients, who now believe that they are invoking a remote object.

In the example above, an object is created as part of the application, after which it is added to an adapter. Effectively, this means that an adapter may need

to support many objects at the same time, leading to potential scalability problems. An alternative solution is to dynamically load objects into memory when they are needed. To do this, Ice provides support for special objects known as *locators*. A locator is called when the adapter receives an incoming request for an object that has not been explicitly added. In that case, the request is forwarded to the locator, whose job is to further handle the request.

To make matters more concrete, suppose a locator is handed a request for an object of which the locator knows that its state is stored in a relational database system. Of course, there is no magic here: the locator has been programmed explicitly to handle such requests. In this case, the object's identifier may correspond to the key of a record in which that state is stored. The locator will then simply do a lookup on that key, fetch the state, and will then be able to further process the request.

There can be more than one locator added to an adapter. In that case, the adapter would keep track of which object identifiers would belong to the same locator. Using multiple locators allows supporting many objects by a single adapter. Of course, objects (or rather their state) would need to be loaded at runtime, but this dynamic behavior would possibly make the server itself relatively simple.

Note 3.7 (Example: Enterprise Java Beans)

The Java programming language and associated model has formed the foundation for numerous distributed systems and applications. Its popularity can be attributed to the straightforward support for object orientation, combined with the inherent support for remote method invocation. Java provides a high degree of access transparency, making it easier to use than, for example, the combination of C with remote procedure calling.

Ever since its introduction, there has been a strong incentive to provide facilities that would ease the development of distributed applications. These facilities go well beyond language support, requiring a runtime environment that supports traditional multitiered client-server architectures. To this end, much work has been put into the development of **(Enterprise) Java Beans (EJB)**.

An EJB is essentially a Java object that is hosted by a special server offering different ways for remote clients to invoke that object. Crucial is that this server provides the support to separate application functionality from systems-oriented functionality. The latter includes functions for looking up objects, storing objects, letting objects be part of a transaction, and so on. How to develop EJBs is described in detail by Schildt [2010].

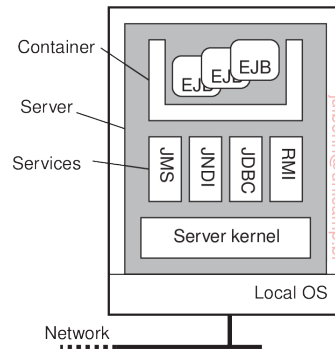


Figure 3.16: General architecture of an EJB server.

With this separation in mind, EJBs can be pictured as shown in Figure 3.16. The important issue is that an EJB is embedded inside a container which effectively provides interfaces to underlying services that are implemented by the application server. The container can more or less automatically bind the EJB to these services, meaning that the correct references are readily available to a programmer. Typical services include those for remote method invocation (RMI), database access (JDBC), naming (JNDI), and messaging (JMS). Making use of these services is more or less automated, but does require that the programmer makes a distinction between four kinds of EJBs:

1. Stateless session beans
2. Stateful session beans
3. Entity beans
4. Message-driven beans

As its name suggests, a **stateless session bean** is an object that is invoked once, does its work, after which it discards any information it needed to perform the service it offered to a client. For example, a stateless session bean could be used to implement a service that lists the top-ranked books. In this case, the bean would typically consist of an SQL query that is submitted to a database. The results would be put into a special format that the client can handle, after which its work would have been completed and the listed books discarded.

In contrast, a **stateful session bean** maintains client-related state. The canonical example is a bean implementing an electronic shopping cart. In this case, a client would typically be able to put things in a cart, remove items, and use the cart to go to an electronic checkout. The bean, in turn, would typically access databases for getting current prices and information on the number of items still in stock. However, its lifetime would still be limited, which is why it is referred to as a session bean: when the client is finished (possibly having invoked the object several times), the bean will automatically be destroyed.

An **entity bean** can be considered to be a long-lived *persistent* object. As such, an entity bean will generally be stored in a database, and likewise, will often also be part of transactions. Typically, entity beans store information that

may be needed a next time a specific client accesses the server. In settings for electronic commerce, an entity bean can be used to record customer information, for example, shipping address, billing address, credit card information, and so on. In these cases, when a client logs in, his associated entity bean will be restored and used for further processing.

Finally, **message-driven beans** are used to program objects that should react to incoming messages (and likewise, be able to send messages). Message-driven beans cannot be invoked directly by a client, but rather fit into a *publish-subscribe* way of communication. What it boils down to is that a message-driven bean is automatically called by the server when a specific message *m* is received, to which the server (or rather an application it is hosting) had previously subscribed, i.e., stated that it wanted to be notified when such a message arrives. The bean contains application code for handling the message, after which the server simply discards it. Message-driven beans are thus seen to be stateless.

Example: The Apache Web server

An interesting example of a server that balances the separation between policies and mechanisms is the Apache Web server. It is also an extremely popular server, estimated to be used to host approximately 50% of all Web sites. Apache is a complex piece of software, and with the numerous enhancements to the types of documents that are now offered in the Web, it is important that the server is highly configurable and extensible, and at the same time largely independent of specific platforms.

Making the server platform independent is realized by essentially providing its own basic runtime environment, which is then subsequently implemented for different operating systems. This runtime environment, known as the **Apache Portable Runtime (APR)**, is a library that provides a platform-independent interface for file handling, networking, locking, threads, and so on. When extending Apache, portability is largely guaranteed provided that only calls to the APR are made and that calls to platform-specific libraries are avoided.

From a certain perspective, Apache can be considered as a completely general server tailored to produce a response to an incoming request. Of course, there are all kinds of hidden dependencies and assumptions by which Apache turns out to be primarily suited for handling requests for Web documents. For example, as we mentioned, Web browsers and servers use HTTP as their communication protocol. HTTP is virtually always implemented on top of TCP, for which reason the core of Apache assumes that all incoming requests adhere to a TCP-based connection-oriented way of communication. Requests based on UDP cannot be handled without modifying the Apache core.

However, the Apache core makes few assumptions on how incoming requests should be handled. Its overall organization is shown in Figure 3.17.

Fundamental to this organization is the concept of a **hook**, which is nothing but a placeholder for a specific group of functions. The Apache core assumes that requests are processed in a number of phases, each phase consisting of a few hooks. Each hook thus represents a group of similar actions that need to be executed as part of processing a request.

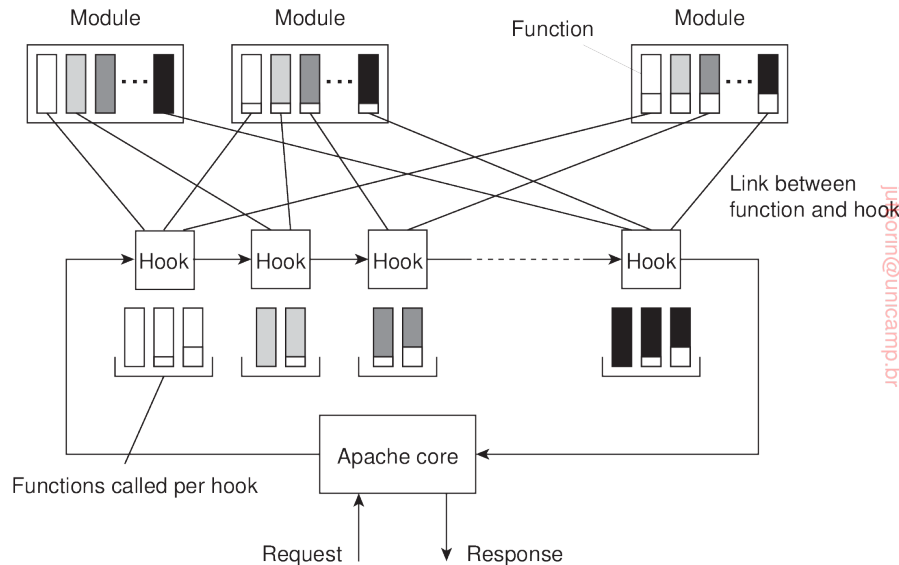


Figure 3.17: The general organization of the Apache Web server.

For example, there is a hook to translate a URL to a local file name. Such a translation will almost certainly need to be done when processing a request. Likewise, there is a hook for writing information to a log, a hook for checking a client's identification, a hook for checking access rights, and a hook for checking which MIME type the request is related to (e.g., to make sure that the request can be properly handled). As shown in Figure 3.17, the hooks are processed in a predetermined order. It is here that we explicitly see that Apache enforces a specific flow of control concerning the processing of requests.

The functions associated with a hook are all provided by separate **modules**. Although, in principle, a developer could change the set of hooks that will be processed by Apache, it is far more common to write modules containing the functions that need to be called as part of processing the standard hooks provided by unmodified Apache. The underlying principle is fairly straightforward. Every hook can contain a set of functions that each should match a specific function prototype (i.e., list of parameters and return type). A module developer will write functions for specific hooks. When compiling Apache, the developer specifies which function should be added to which hook. The latter is shown in Figure 3.17 as the various links between functions

and hooks.

Because there may be tens of modules, each hook will generally contain several functions. Normally, modules are considered to be mutual independent, so that functions in the same hook will be executed in some arbitrary order. However, Apache can also handle module dependencies by letting a developer specify an ordering in which functions from different modules should be processed. By and large, the result is a Web server that is extremely versatile. Detailed information on configuring Apache, as well as an introduction to how it can be extended is found in [Laurie and Laurie, 2002].

Server clusters

In Chapter 1, we briefly discussed cluster computing as one of the many appearances of distributed systems. We now take a closer look at the organization of server clusters, along with the salient design issues. We first consider common server clusters that are organized in local-area networks. A special group is formed by wide-area server clusters, which we subsequently discuss.

Local-area clusters

Simply put, a server cluster is nothing else but a collection of machines connected through a network, where each machine runs one or more servers. The server clusters that we consider here, are the ones in which the machines are connected through a local-area network, often offering high bandwidth and low latency.

General organization In many cases, a server cluster is logically organized into three tiers, as shown in Figure 3.18. The first tier consists of a (logical) switch through which client requests are routed. Such a switch can vary widely. For example, transport-layer switches accept incoming TCP connection requests and pass requests on to one of servers in the cluster. A completely different example is a Web server that accepts incoming HTTP requests, but that partly passes requests to application servers for further processing only to later collect results from those servers and return an HTTP response.

As in any multitiered client-server architecture, many server clusters also contain servers dedicated to application processing. In cluster computing, these are typically servers running on high-performance hardware dedicated to delivering compute power. However, in the case of enterprise server clusters, it may be the case that applications need only run on relatively low-end machines, as the required compute power is not the bottleneck, but access to storage is.

This brings us the third tier, which consists of data-processing servers, notably file and database servers. Again, depending on the usage of the server

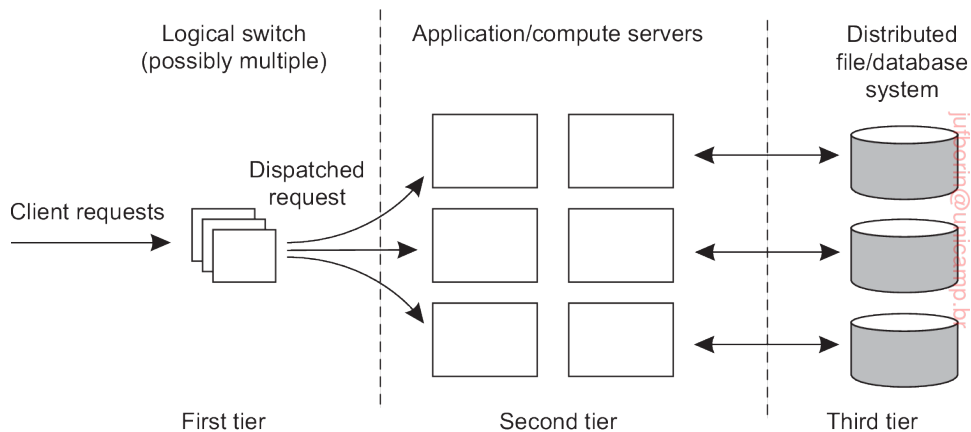


Figure 3.18: The general organization of a three-tiered server cluster.

cluster, these servers may be running on specialized machines, configured for high-speed disk access and having large server-side data caches.

Of course, not all server clusters will follow this strict separation. It is frequently the case that each machine is equipped with its own local storage, often integrating application and data processing in a single server leading to a two-tiered architecture. For example, when dealing with streaming media by means of a server cluster, it is common to deploy a two-tiered system architecture, where each machine acts as a dedicated media server [Steinmetz and Nahrstedt, 2004].

When a server cluster offers multiple services, it may happen that different machines run different application servers. As a consequence, the switch will have to be able to distinguish services or otherwise it cannot forward requests to the proper machines. As a consequence, we may find that certain machines are temporarily idle, while others are receiving an overload of requests. What would be useful is to temporarily migrate services to idle machines. A solution is to use virtual machines allowing a relatively easy migration of code to real machines.

Request dispatching Let us now take a closer look at the first tier, consisting of the switch, also known as the **front end**. An important design goal for server clusters is to hide the fact that there are multiple servers. In other words, client applications running on remote machines should have no need to know anything about the internal organization of the cluster. This access transparency is invariably offered by means of a single access point, in turn implemented through some kind of hardware switch such as a dedicated machine.

The switch forms the entry point for the server cluster, offering a single network address. For scalability and availability, a server cluster may have

multiple access points, where each access point is then realized by a separate dedicated machine. We consider only the case of a single access point.

A standard way of accessing a server cluster is to set up a TCP connection over which application-level requests are then sent as part of a session. A session ends by tearing down the connection. In the case of **transport-layer switches**, the switch accepts incoming TCP connection requests, and hands off such connections to one of the servers. There are essentially two ways how the switch can operate [Cardellini et al., 2002].

In the first case, the client sets up a TCP connection such that all requests and responses pass through the switch. The switch, in turn, will set up a TCP connection with a selected server and pass client requests to that server, and also accept server responses (which it will pass on to the client). In effect, the switch sits in the middle of a TCP connection between the client and a selected server, rewriting the source and destination addresses when passing TCP segments. This approach is a form of **network-address translation (NAT)** [Srisuresh and Holdrege, 1999].

Alternatively, the switch can actually hand off the connection to a selected server such that all responses are directly communicated to the client without passing through the switch [Hunt et al., 1997; Pai et al., 1998]. The principle working of what is commonly known as **TCP handoff** is shown in Figure 3.19.

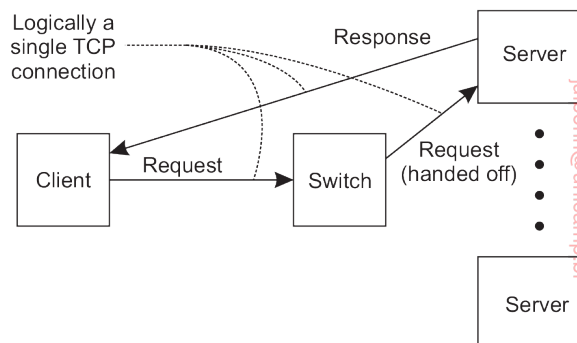


Figure 3.19: The principle of TCP handoff.

When the switch receives a TCP connection request, it first identifies the best server for handling that request, and forwards the request packet to that server. The server, in turn, will send an acknowledgment back to the requesting client, but inserting the switch's IP address as the source field of the header of the IP packet carrying the TCP segment. Note that this address rewriting is necessary for the client to continue executing the TCP protocol: it is expecting an answer back from the switch, not from some arbitrary server it has never heard of before. Clearly, a TCP-handoff implementation requires operating-system level modifications. TCP handoff is especially effective when responses are much larger than requests, as in the case of Web servers.

It can already be seen that the switch can play an important role in distributing the load among the various servers. By deciding where to forward a request to, the switch also decides which server is to handle further processing of the request. The simplest load-balancing policy that the switch can follow is round robin: each time it picks the next server from its list to forward a request to. Of course, the switch will have to keep track to which server it handed off a TCP connection, at least until that connection is torn down. As it turns out, maintaining this state and handing off subsequent TCP segments belonging to the same TCP connection, may actually slow down the switch.

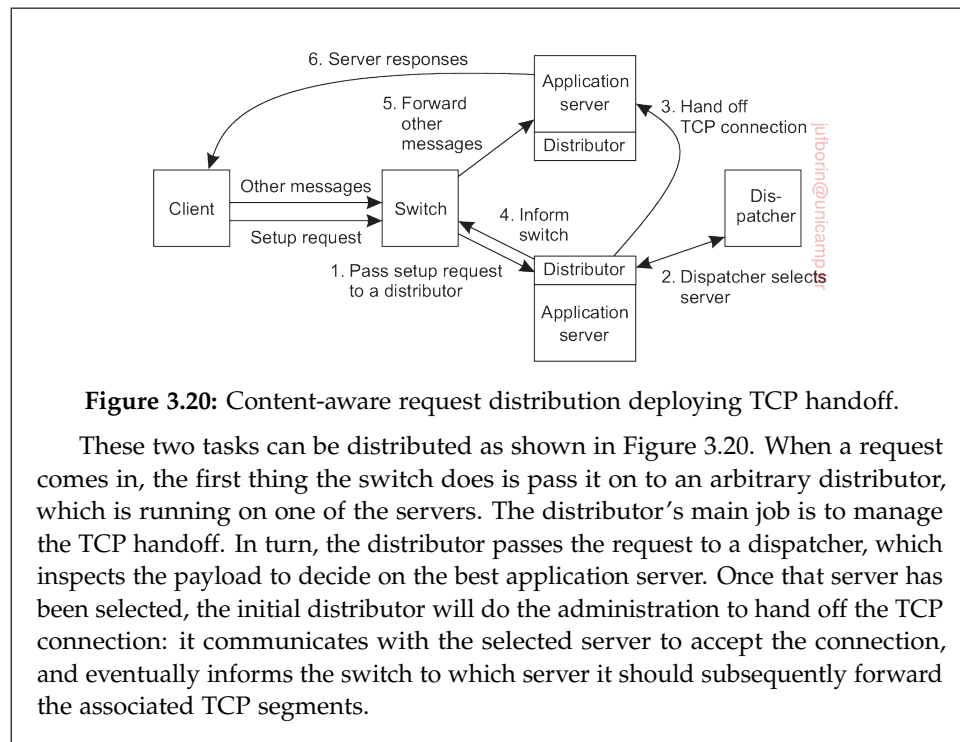
More advanced server selection criteria can be deployed as well. For example, assume multiple services are offered by the server cluster. If the switch can distinguish those services when a request comes in, it can then take informed decisions on where to forward the request to. This server selection can still take place at the transport level, provided services are distinguished by means of a port number. In the case of **transport-level switches**, as we have discussed so far, decisions on where to forward an incoming request is based on transport-level information only. One step further is to have the switch actually inspect the payload of the incoming request. This **content-aware request distribution** can be applied only if it is known what that payload looks like. For example, in the case of Web servers, the switch can expect an HTTP request, based on which it can then decide who is to process it.

Note 3.8 (Advanced: Efficient content-aware request distribution)

Obviously, the design of the switch is crucial, as it can easily become a bottleneck with all traffic passing through it. Transport-level switches are generally very efficient. However, content-aware switches operating at the level of the application layer that inspect the payload of an incoming request, may actually need to do a lot of processing.

Content-aware request distribution has several advantages. For example, if the switch always forwards requests for the same file to the same server, that server may be able to effectively cache the file resulting in higher response times. Likewise, we may decide to distribute a set of files to specific servers, making a distinction, for example, between streaming media files, images, text files, but also perhaps actual databases. Being able to make such a distinction will allow to install dedicated servers in the second tier.

Ideally, the switch would be as efficient as a normal transport-level switch, yet have the functionality for content-aware distribution. Aron et al. [2000] propose a scheme by which the work for inspecting the payload of an incoming request is distributed across several servers, and combine this distribution with transport-level switching. The switch now has two tasks. First, when a request initially comes in, it must decide which server will handle the rest of the communication with the client. Second, the switch should forward the client's TCP messages associated with the handed-off TCP connection.



Wide-area clusters

A characteristic feature of local-area server clusters is that they are owned by a single organization. Deploying clusters across a wide-area network has traditionally been quite cumbersome as one had to generally deal with multiple administrative organizations such as ISPs (Internet Service Providers). With the advent of cloud computing, matters have changed and we are now witnessing an increase of wide-area distributed systems in which servers (or server clusters) are spread across the Internet. The problems related to having to deal with multiple organizations are effectively circumvented by making use of the facilities of a single cloud provider.

Cloud providers like Amazon and Google manage several data centers placed at different locations worldwide. As such, they can offer an end user the ability to build a wide-area distributed system consisting of a potentially large collection of networked virtual machines, scattered across the Internet. An important reason for wanting such distributed systems is to provide locality: offering data and services that are close to clients. An example where such locality is important is streaming media: the closer a video server is located to a client, the easier it becomes to provide high-quality streams.

Note that if wide-area locality is not critical, it may suffice, or even be better, to place virtual machines in a single data center, so that interprocess

communication can benefit from low-latency local networks. The price to pay may be higher latencies between clients and the service running in a remote data center.

Request dispatching If wide-area locality is an issue, then request dispatching becomes important: if a client accesses a service, its request should be forwarded to a nearby server, that is, a server that will allow communication with that client to be fast. Deciding which server should handle the client's request is an issue of **redirection policy** [Sivasubramanian et al., 2004b]. If we assume that a client will initially contact a request dispatcher analogous to the switch in our discussion of local-area clusters, then that dispatcher will have to estimate the latency between the client and several servers. How such an estimation can be made is discussed in Section 6.5.

Once a server has been selected, the dispatcher will have to inform the client. Several **redirection mechanisms** are possible. A popular one is when the dispatcher is actually a DNS name server. Internet or Web-based services are often looked up in the **Domain Name System (DNS)**. A client provides a domain name such as `service.organization.org` to a local DNS server, which eventually returns an IP address of the associated service, possibly after having contacted other DNS servers. When sending its request to look up a name, a client also sends its own IP address (DNS requests are sent as UDP packets). In other words, the DNS server will also know the client's IP address which it can then subsequently use to select the best server for that client, and returning a close-by IP address.

Unfortunately, this scheme is not perfect for two reasons. First, rather than sending the client's IP address, what happens is that the local DNS server that is contacted by the client acts as a proxy for that client. In other words, not the client's IP address, but that of the local DNS server is used to identify the location of the client. Mao et al. [2002] have shown that there may be a huge additional communication cost, as the local DNS server is often not *that* local.

Secondly, depending on the scheme that is used for resolving a domain name, it may even be the case that the address of the local DNS server is not even being used. Instead, it may happen that the DNS server that is deciding on which IP address to return, may be fooled by the fact that the requester is yet another DNS server acting as an intermediate between the original client and the deciding DNS server. In those cases, locality awareness has been completely lost.

Despite that DNS-based redirection may not always be very accurate, it is widely deployed if only for the fact that it is relatively easy to implement and also transparent to the client. In addition, there is no need to rely on location-aware client-side software.

Note 3.9 (Advanced: An alternative for organizing wide-area server clusters)

As we mentioned, most server clusters offer a single access point. When that point fails, the cluster becomes unavailable. To eliminate this potential problem, several access points can be provided, of which the addresses are made publicly available. The **Domain Name System (DNS)** can return several addresses, all belonging to the same host name, using a simple round-robin strategy. This approach still requires clients to make several attempts if one of the addresses fails. Moreover, this does not solve the problem of requiring static access points.

Having stability, like a long-living access point, is a desirable feature from a client's and a server's perspective. On the other hand, it is also desirable to have a high degree of flexibility in configuring a server cluster, including the switch. This observation has led to a design of a **distributed server** which effectively is nothing but a possibly dynamically changing set of machines, with also possibly varying access points, but which nevertheless appears to the outside world as a single, powerful machine. Szymaniak et al. [2007] provide the design of such a distributed server. We describe it briefly here.

The basic idea behind a distributed server is that clients benefit from a robust, high-performing, stable server. These properties can often be provided by high-end mainframes, of which some have an acclaimed mean time between failure of more than 40 years. However, by grouping simpler machines transparently into a cluster, and not relying on the availability of a single machine, it may be possible to achieve a better degree of stability than by each component individually. For example, such a cluster could be dynamically configured from end-user machines, as in the case of a collaborative distributed system. Note also that many data centers are making use of relatively cheap and simple machines [Barroso and Hölze, 2009].

Let us concentrate on how a stable access point can be achieved in such a system. The main idea is to make use of available networking services, notably mobility support for IP version 6 (MIPv6). In MIPv6, a mobile node is assumed to have a **home network** where it normally resides and for which it has an associated stable address, known as its **home address (HoA)**. This home network has a special router attached, known as the **home agent**, which will take care of traffic to the mobile node when it is away. To this end, when a mobile node attaches to a foreign network, it will receive a temporary **care-of address (CoA)** where it can be reached. This care-of address is reported to the node's home agent who will then see to it that all traffic is forwarded to the mobile node. Note that applications communicating with the mobile node will see only the address associated with the node's home network. They will never see the care-of address.

This principle can be used to offer a stable address of a distributed server. In this case, a single unique **contact address** is initially assigned to the server cluster. The contact address will be the server's life-time address to be used in all communication with the outside world. At any time, one node in the distributed server will operate as an access point using that contact address, but this role can easily be taken over by another node. What happens is that the access point records its own address as the care-of address at the home agent associated with the distributed server. At that point, all traffic will be directed to the access point,

who will then take care of distributing requests among the currently participating nodes. If the access point fails, a simple fail-over mechanism comes into place by which another access point reports a new care-of address.

This simple configuration would make the home agent as well as the access point a potential bottleneck as all traffic would flow through these two machines. This situation can be avoided by using an MIPv6 feature known as **route optimization**. Route optimization works as follows. Whenever a mobile node with home address HA reports its current care-of address, say CA, the home agent can forward CA to a client. The latter will then locally store the pair (HA, CA). From that moment on, communication will be directly forwarded to CA. Although the application at the client side can still use the home address, the underlying support software for MIPv6 will translate that address to CA and use that instead.

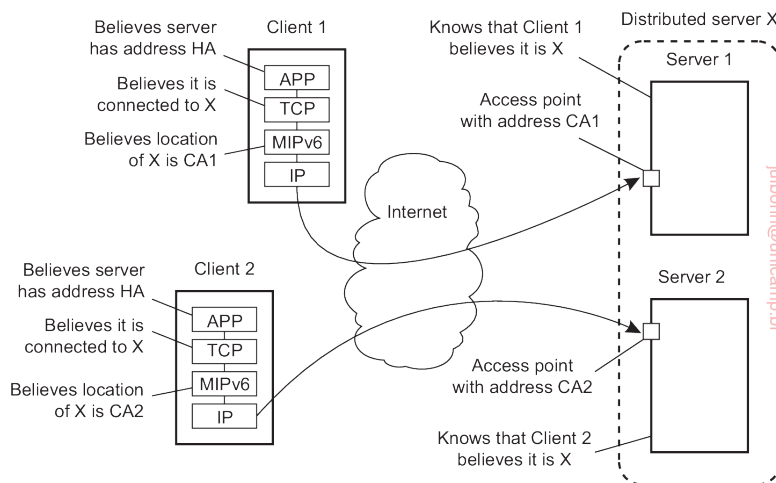


Figure 3.21: Route optimization in a distributed server.

Route optimization can be used to make different clients believe they are communicating with a single server, where, in fact, each client is communicating with a different member node of the distributed server, as shown in Figure 3.21. To this end, when an access point of a distributed server forwards a request from client C_1 to, say node S_1 (with address CA_1), it passes enough information to S_1 to let it initiate the route optimization procedure by which eventually the client is made to believe that the care-of address is CA_1 . This will allow C_1 to store the pair (HA, CA_1). During this procedure, the access point (as well as the home agent) tunnel most of the traffic between C_1 and S_1 . This will prevent the home agent from believing that the care-of address has changed, so that it will continue to communicate with the access point.

Of course, while this route optimization procedure is taking place, requests from other clients may still come in. These remain in a pending state at the access point until they can be forwarded. The request from another client C_2 may then be forwarded to member node S_2 (with address CA_2), allowing the latter to let client C_2 store the pair (HA, CA_2). As a result, different clients will be

directly communicating with different members of the distributed server, where each client application still has the illusion that this server has address HA. The home agent continues to communicate with the access point talking to the contact address.

Case study: PlanetLab

Let us now take a closer look at a somewhat unusual cluster server. PlanetLab is a collaborative distributed system in which different organizations each donate one or more computers, adding up to a total of hundreds of nodes. Together, these computers form a 1-tier server cluster, where access, processing, and storage can all take place on each node individually. Management of PlanetLab is by necessity almost entirely distributed.

General organization In PlanetLab, a participating organization donates one or more nodes (i.e., computers) that are subsequently shared among all PlanetLab users. Each node is organized as shown in Figure 3.22. There are two important components [Bavier et al., 2004; Peterson et al., 2006]. The first one is the virtual machine monitor (VMM), which is an enhanced Linux operating system. The enhancements mainly comprise adjustments for supporting the second component, namely (Linux) **Vservers**. For now, a Vserver can best be thought of as a separate environment in which a group of processes run. We return to Vservers below.

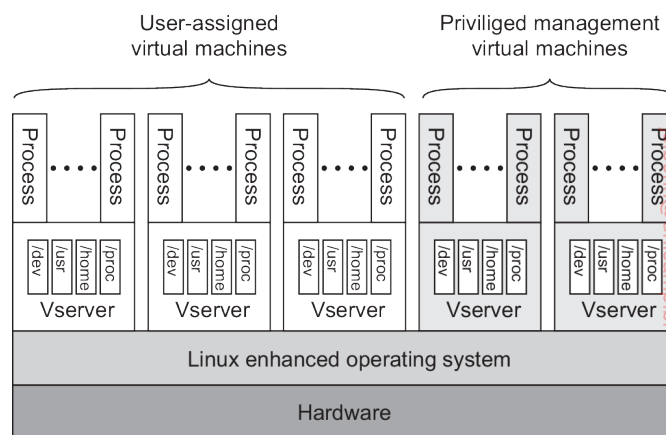


Figure 3.22: The basic organization of a PlanetLab node.

The Linux VMM ensures that Vservers are separated: processes in different Vservers are executed concurrently and independently, each making use only of the software packages and programs available in their own environment. The isolation between processes in different Vservers is strict. For example,

two processes in different Vservers may have the same user ID, but this does not imply that they stem from the same user. This separation considerably eases supporting users from different organizations that want to use PlanetLab as, for example, a testbed to experiment with completely different distributed systems and applications.

To support such experimentations, PlanetLab uses **slices**, each slice being a set of Vservers, each Vserver running on a different node, as illustrated in Figure 3.23. A slice can thus be thought of as a virtual server cluster, implemented by means of a collection of virtual machines.

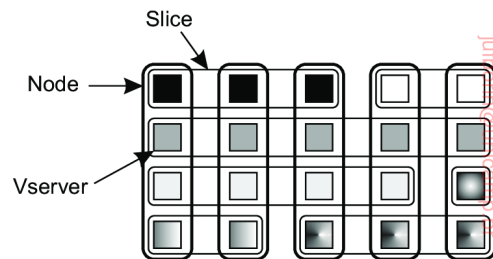


Figure 3.23: The principle of a PlanetLab slice, showing sets of associated Vservers across different nodes.

Central to managing PlanetLab resources is the **node manager**. Each node has such a manager, implemented by means of a separate Vserver, whose only task is to create other Vservers on the node it manages and to control resource allocation. To create a new slice, each node will also run a **slice creation service (SCS)**, which, in turn, can contact the node manager requesting it to create a Vserver and to allocate resources. The node manager itself cannot be contacted directly over a network, allowing it to concentrate only on local resource management. In turn, the SCS will not accept slice-creation requests from just anybody. Only specific **slice authorities** are eligible for requesting the creation of a slice. Each slice authority will have access rights to a collection of nodes. The simplest model is that there is only a single, centralized slice authority that is allowed to request slice creation on all nodes. In practice, we see that this slice authority is the one used to get a user up-and-running on PlanetLab.

Keeping track of resources is done by means of a resource specification, or *rspec* for short. An *rspec* specifies a time interval during which certain resources have been allocated. Resources include disk space, file descriptors, inbound and outbound network bandwidth, transport-level end points, main memory, and CPU usage. An *rspec* is identified through a globally unique 128-bit identifier known as a resource capability (*rcap*). Given an *rcap*, the node manager can look up the associated *rspec* in a local table.

Resources are bound to slices. In other words, in order to make use of resources, it is necessary to create a slice. Each slice is associated with a

service provider, which can best be seen as an entity having an account on PlanetLab. Every slice can then be identified by a (principal_id, slice_tag) pair, where the principal_id identifies the provider and slice_tag is an identifier chosen by the provider.

Vservers Let us now turn our attention to PlanetLab's Vservers, which have been described and evaluated by Soltesz et al. [2007]. A Vserver is organized according to what is called a **container-based approach**. The main difference with traditional virtual machines as those discussed in Section 3.2, is that they rely on a single, shared operating-system kernel. This also means that resource management is mostly done only by the underlying operating system and not by any of the Vservers. The primary task of a Vserver is therefore to merely support a group of processes and keep that group isolated from processes running under the jurisdiction of another Vserver. Indeed, a Vserver forms an isolated container for a group of processes and their allocated resources. By now, containers have become quite popular for offering cloud services.

This isolation is technically established by the underlying VMM, for which purpose the Linux operating system has been adapted. One of the most salient adaptations concerns the separation of independent name spaces. For example, to create the illusion that a Vserver is really a single machine, the Unix process init traditionally always gets process ID 1 (with its parent having ID 0). Obviously, the Linux VMM will already have such a process running, yet it needs to create another init process for every Vserver. Those processes will each also get process ID 1, while the kernel keeps track of a mapping between such a virtual process ID and the real, actually assigned process ID. Other examples of consistent naming across Vservers easily come to mind.

Likewise, isolation is also established by providing each Vserver with its own set of libraries, yet using the directory structure that those libraries expect, that is, with directories named /dev, /home, /proc, /usr, and so on (which is also depicted in Figure 3.22). In principle, such a separated name space can be achieved using the standard chroot command, effectively giving each Vserver its own root directory. However, special measures are needed at kernel level to prevent unauthorized access of one Vserver to the directory tree of another Vserver, as explained by Soltesz et al. [2007].

An important advantage of the container-based approach toward virtualization in comparison to running separate guest operating systems, is that resource allocation can generally be much simpler. In particular, it is possible to overbook resources by allowing for dynamic resource allocation, just as is done with allocating resources to normal processes. Normally, when using a guest operating system, the guest will have to be allocated a fixed amount of resources in advance (notably main memory). When considering that the nodes provided by participating PlanetLab organizations are required to have only few GByte of main memory, it is not hard to imagine that memory

may be a scarce resource. It is therefore necessary to dynamically allocate memory to allow tens of virtual machines to be running at the same time on a single node. Vservers are ideal for this type of resource management; operating systems are much harder to support in such cases. Of course, this cannot prevent a Vserver from using too much memory on a busy node. The PlanetLab policy in that case is simple: the Vserver hogging memory when swap space is almost filled, is reset.

3.5 Code migration

So far, we have been mainly concerned with distributed systems in which communication is limited to passing data. However, there are situations in which passing programs, sometimes even while they are being executed, simplifies the design of a distributed system. In this section, we take a detailed look at what code migration actually is. We start by considering different approaches to code migration, followed by a discussion on how to deal with the local resources that a migrating program uses. A particularly hard problem is migrating code in heterogeneous systems, which is also discussed.

Reasons for migrating code

Traditionally, code migration in distributed systems took place in the form of **process migration** in which an entire process was moved from one node to another [Milojicic et al., 2000]. Moving a running process to a different machine is a costly and intricate task, and there had better be a good reason for doing so. That reason has always been performance. The basic idea is that overall system performance can be improved if processes are moved from heavily loaded to lightly loaded machines. Load is often expressed in terms of the CPU queue length or CPU utilization, but other performance indicators are used as well. When completing their survey, Milojicic et al. had already come to the conclusion that process migration was no longer a viable option for improving distributed systems.

However, instead of offloading machines, we can now witness that code is moved to make sure that a machine is *sufficiently* loaded. In particular, migrating complete virtual machines with their suite of applications to lightly loaded machines in order to minimize the total number of nodes being used is common practice in optimizing energy usage in data centers. Interestingly enough, although migrating virtual machines may require more resources, the task itself is far less intricate than migrating a process, as we discuss in Note 3.11.

In general, load-distribution algorithms by which decisions are made concerning the allocation and redistribution of tasks with respect to a set of machines, play an important role in compute-intensive systems. However, in many modern distributed systems, optimizing computing capacity is less an

issue than, for example, trying to minimize communication. Moreover, due to the heterogeneity of the underlying platforms and computer networks, performance improvement through code migration is often based on qualitative reasoning instead of mathematical models.

Consider, as an example, a client-server system in which the server manages a huge database. If a client application needs to perform many database operations involving large quantities of data, it may be better to ship part of the client application to the server and send only the results across the network. Otherwise, the network may be swamped with the transfer of data from the server to the client. In this case, code migration is based on the assumption that it generally makes sense to process data close to where those data reside.

This same reason can be used for migrating parts of the server to the client. For example, in many interactive database applications, clients need to fill in forms that are subsequently translated into a series of database operations. Processing the form at the client side, and sending only the completed form to the server, can sometimes avoid that a relatively large number of small messages need to cross the network. The result is that the client perceives better performance, while at the same time the server spends less time on form processing and communication. In the case of smartphones, moving code to be executed at the handheld instead of the server may be the only viable solution to obtain acceptable performance, *both* for the client and the server (see Kumar et al. [2013] for a survey on offloading computations).

Support for code migration can also help improve performance by exploiting parallelism, but without the usual intricacies related to parallel programming. A typical example is searching for information in the Web. It is relatively simple to implement a search query in the form of a small mobile program, called a **mobile agent**, that moves from site to site. By making several copies of such a program, and sending each off to different sites, we may be able to achieve a linear speed-up compared to using just a single program instance. However, Carzaniga et al. [2007] conclude that mobile agents have never become successful because they did not really offer an obvious advantage over other technologies. Moreover, and crucial, it turned out to be virtually impossible to let this type of mobile code operate in a secure way.

Besides improving performance, there are other reasons for supporting code migration as well. The most important one is that of flexibility. The traditional approach to building distributed applications is to partition the application into different parts, and decide in advance where each part should be executed. This approach, for example, has lead to different multitiered client-server applications discussed in Section 2.3.

However, if code can move between different machines, it becomes possible to dynamically configure distributed systems. For example, suppose a server

implements a standardized interface to a file system. To allow remote clients to access the file system, the server makes use of a proprietary protocol. Normally, the client-side implementation of the file system interface, which is based on that protocol, would need to be linked with the client application. This approach requires that the software be readily available to the client at the time the client application is being developed.

An alternative is to let the server provide the client's implementation no sooner than is strictly necessary, that is, when the client binds to the server. At that point, the client dynamically downloads the implementation, goes through the necessary initialization steps, and subsequently invokes the server. This principle is shown in Figure 3.24 (we note that the code repository is generally located as part of the server). This model of dynamically moving code from a remote site does require that the protocol for downloading and initializing code is standardized. Also, it is necessary that the downloaded code can be executed on the client's machine. Typically, scripts that run in a virtual machine embedded in, for example, a Web browser, will do the trick. Arguably, this form of code migration has been key to the success of the dynamic Web. These and other solutions are discussed below and in later chapters.

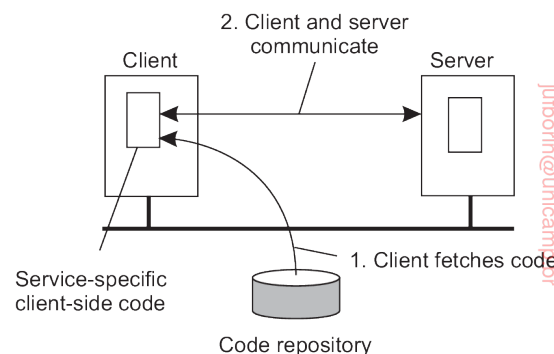


Figure 3.24: The principle of dynamically configuring a client to communicate with a server.

The important advantage of this model of dynamically downloading client-side software is that clients need not have all the software preinstalled to talk to servers. Instead, the software can be moved in as necessary, and likewise, discarded when no longer needed. Another advantage is that as long as interfaces are standardized, we can change the client-server protocol and its implementation as often as we like. Changes will not affect existing client applications that rely on the server.

There are, of course, also disadvantages. The most serious one, which we discuss in Chapter 9, has to do with security. Blindly trusting that the downloaded code implements only the advertised interface while accessing

your unprotected hard disk and does not send the juiciest parts to heaven-knows-who may not always be such a good idea. Fortunately, it is well understood how to protect the client against malicious, downloaded code.

Note 3.10 (More information: Moving away from thin-client computing?)

By now, there is much more insight and expertise concerning transparent and safe dynamic migration of code to clients. As a result, the trend that we described in Note 2.4 of moving toward thin-client computing because managing client-side software often turned out to be cumbersome, has been partly reverted. By dynamically migrating client-side software, yet keeping the management of that software entirely at the server side (or rather, at its owner), having “richer” client-side software has become practically feasible.

Note 3.11 (Advanced: Models for code migration)

Although code migration suggests that we move only code between machines, the term actually covers a much richer area. Traditionally, communication in distributed systems is concerned with exchanging data between processes. Code migration in the broadest sense deals with moving programs between machines, with the intention to have those programs be executed at the target. In some cases, as in process migration, the execution status of a program, pending signals, and other parts of the environment must be moved as well.

To get a better understanding of the different models for code migration, we use a framework proposed by Fuggetta et al. [1998]. In this framework, a process consists of three segments. The **code segment** is the part that contains the set of instructions that make up the program that is being executed. The **resource segment** contains references to external resources needed by the process, such as files, printers, devices, other processes, and so on. Finally, an **execution segment** is used to store the current execution state of a process, consisting of private data, the stack, and, of course, the program counter.

A further distinction can be made between sender-initiated and receiver-initiated migration. In **sender-initiated** migration, migration is initiated at the machine where the code currently resides or is being executed. Typically, sender-initiated migration is done when uploading programs to a compute server. Another example is sending a query, or batch of queries, to a remote database server. In **receiver-initiated** migration, the initiative for code migration is taken by the target machine. Java applets are an example of this approach.

Receiver-initiated migration is simpler than sender-initiated migration. In many cases, code migration occurs between a client and a server, where the client takes the initiative for migration. Securely uploading code to a server, as is done in sender-initiated migration, often requires that the client has previously been registered and authenticated at that server. In other words, the server is required to know all its clients, the reason being is that the client will presumably want access to the server’s resources such as its disk. Protecting such resources is essential. In contrast, downloading code as in the receiver-initiated case, can often

be done anonymously. Moreover, the server is generally not interested in the client's resources. Instead, code migration to the client is done only for improving client-side performance. To that end, only a limited number of resources need to be protected, such as memory and network connections. We return to secure code migration extensively in Chapter 9.

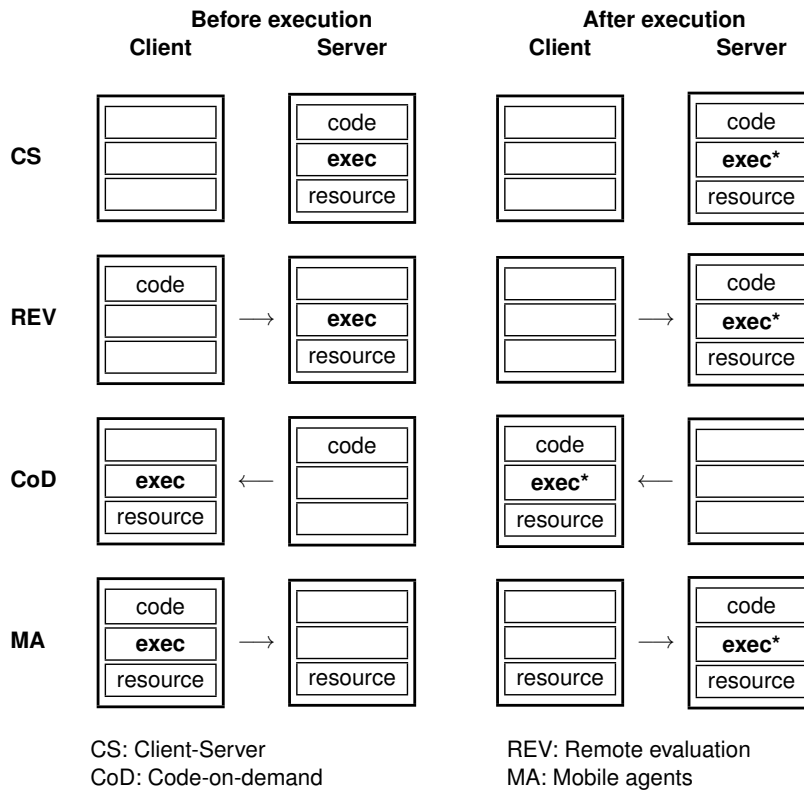


Figure 3.25: Four different paradigms for code mobility.

This brings us to four different paradigms for code mobility, as shown in Figure 3.25. Following Fuggetta et al. [1998], we make a distinction between simple **client-server computing**, **remote evaluation**, **code-on-demand**, and **mobile agents**. Figure 3.25 shows the situation at respectively the client and the server, before and after execution of the mobile code.

In the case of client-server computing, the code, execution state, and resource segment are all located at the server, and after execution, only the execution state at the server is generally modified. This state modification is denoted by means of an asterisk. With the sender-initiated **remote evaluation**, the client migrates code to the server where that code is executed and leading to a modification of the execution state at the server. **Code-on-demand** is a receiver-initiated scheme by which the client obtains code from the server with its execution modifying the client-side execution state and operating on the client's resources. Finally, **mobile agents** typically follow a sender-initiated approach, moving code as well

as execution state from the client to the server, operating on both the client's as well as the server's resources. The execution of a mobile agent will generally lead to modification of the associated execution state.

The bare minimum for code migration is to provide only **weak mobility**. In this model, it is possible to transfer only the code segment, along with perhaps some initialization data. A characteristic feature of weak mobility is that a transferred program is always started anew. This is what happens, for example, with Java applets, which start from the same initial state. In other words, no history from where the migrated code left off at a previous location is maintained by the underlying middleware. If such history needs to be preserved, it will have to be encoded as part of the mobile application itself. The benefit of weak mobility is its simplicity, as it requires only that the target machine can execute the code segment. In essence, this boils down to making the code portable. We return to these matters when discussing migration in heterogeneous systems.

In contrast to weak mobility, in systems that support **strong mobility** the execution segment can be transferred as well. The characteristic feature of strong mobility is that a running process can be stopped, subsequently moved to another machine, and then resume execution exactly where it left off. Clearly, strong mobility is much more general than weak mobility, but also much more difficult to implement. In particular, when migrating a process, the execution segment generally also contains data that is highly dependent on a specific *implementation* of the underlying operating system. For example, it may rely on information normally found in the operating system's process table. As a consequence, migrating to a different operating system, even one that belongs to the same family as the source, may cause a lot of headaches.

In the case of weak mobility, it also makes a difference if the migrated code is executed by the target process, or whether a separate process is started. For example, Java applets are simply downloaded by a Web browser and are executed in the browser's address space. The benefit of this approach is that there is no need to start a separate process, thereby avoiding interprocess communication at the target machine. The main drawback, obviously, is that the target process needs to be protected against malicious or inadvertent code executions, which may be reason enough to isolate the migrated code in a separate process.

Instead of moving a running process, also referred to as process migration, strong mobility can also be supported by remote cloning. In contrast to process migration, cloning yields an exact copy of the original process, but now running on a different machine. The cloned process is executed in parallel to the original process. In Unix systems, remote cloning takes place by forking off a child process and letting that child continue on a remote machine. The benefit of cloning is that the model closely resembles the one that is already used in many applications. The only difference is that the cloned process is executed on a different machine. In this sense, migration by cloning is a simple way to improve distribution transparency.

Migration in heterogeneous systems

So far, we have tacitly assumed that the migrated code can be easily executed at the target machine. This assumption is in order when dealing with homogeneous systems. In general, however, distributed systems are constructed on a heterogeneous collection of platforms, each having their own operating system and machine architecture.

The problems coming from heterogeneity are in many respects the same as those of portability. Not surprisingly, solutions are also very similar. For example, at the end of the 1970s, a simple solution to alleviate many of the problems of porting Pascal to different machines was to generate machine-independent intermediate code for an abstract virtual machine [Barron, 1981]. That machine, of course, would need to be implemented on many platforms, but it would then allow Pascal programs to be run anywhere. Although this simple idea was widely used for some years, it never really caught on as the general solution to portability problems for other languages, notably C.

About 25 years later, code migration in heterogeneous systems is being tackled by scripting languages and highly portable languages such as Java. In essence, these solutions adopt the same approach as was done for porting Pascal. All such solutions have in common that they rely on a (process) virtual machine that either directly interprets source code (as in the case of scripting languages), or otherwise interprets intermediate code generated by a compiler (as in Java). Being in the right place at the right time is also important for language developers.

Further developments have weakened the dependency on programming languages. In particular, solutions have been proposed to migrate not only processes, but to migrate entire computing environments. The basic idea is to compartmentalize the overall environment and to provide processes in the same part their own view on their computing environment. That compartmentalization takes place in the form of virtual machine monitors running an operating system and a suite of applications.

With virtual machine migration, it becomes possible to decouple a computing environment from the underlying system and actually migrate it to another machine (see Medina and Garcia [2014] for an overview on migration mechanisms for virtual machines). A major advantage of this approach is that processes can remain ignorant of the migration itself: they need not be interrupted in their execution, nor should they experience any problems with used resources. The latter are either migrating along with a process, or the way that a process accesses a resource is left unaffected (at least, for that process).

As an example, Clark et al. [2005] concentrated on real-time migration of a virtualized operating system, typically something that would be convenient in a cluster of servers where a tight coupling is achieved through a single, shared local-area network. Under these circumstances, migration involves two

major problems: migrating the entire memory image and migrating bindings to local resources.

As to the first problem, there are, in principle, three ways to handle migration (which can be combined):

1. Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
2. Stopping the current virtual machine; migrate memory, and start the new virtual machine.
3. Letting the new virtual machine pull in new pages as needed, that is, let processes start on the new virtual machine immediately and copy memory pages on demand.

The second option may lead to unacceptable downtime if the migrating virtual machine is running a live service, that is, one that offers continuous service. On the other hand, a pure on-demand approach as represented by the third option may extensively prolong the migration period, but may also lead to poor performance because it takes a long time before the working set of the migrated processes has been moved to the new machine.

As an alternative, Clark et al. [2005] propose to use a pre-copy approach which combines the first option, along with a brief stop-and-copy phase as represented by the second option. As it turns out, this combination can lead to very low service downtimes (see also Note 3.12).

Concerning local resources, matters are simplified when dealing only with a cluster server. First, because there is a single network, the only thing that needs to be done is to announce the new network-to-MAC address binding, so that clients can contact the migrated processes at the correct network interface. Finally, if it can be assumed that storage is provided as a separate tier (like we showed in Figure 3.18), then migrating binding to files is similarly simple, as it effectively means reestablishing network connections.

Note 3.12 (Advanced: On the performance of live virtual machine migration)

One potential problem with virtual-machine migration is that it may take considerable time. This by itself need not be bad as long as the services that are running on the migrating virtual machine can continue to operate. An approach used in practice was briefly described above. First, memory pages are copied to the target machine, possibly sending updates of pages that were modified while copying took place (remember that copying lots of memory may take tens of seconds, even across a high-speed local network). Second, when most pages have been faithfully copied, the current machine is stopped, the remaining dirty pages are copied to the target, where the now exact copy can then be started where the original left off.

The downtime in which the remaining dirty pages need to be copied depends on the applications running on the virtual machine. Clark et al. [2005] report

downtimes for specific configurations between 60 msecs and less than 4 secs. Voorsluys et al. [2009] come to similar values. However, what may be more interesting is to observe what the response time is of the service running on the virtual machine while the latter is being migrated. The model in this case is that the service continues to operate on the original machine until full migration has completed. However, we cannot ignore that migration itself is a resource-intensive operation, requiring considerable processing capacity as well as network bandwidth.

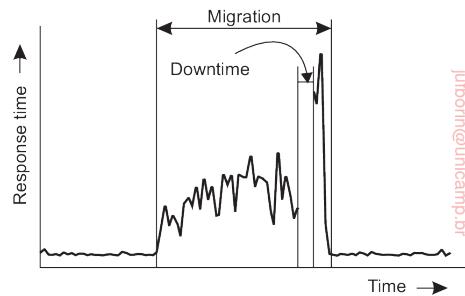


Figure 3.26: The effect on the response time of a service while migrating its underlying virtual machine. Adapted from Voorsluys et al. [2009].

Voorsluys et al. [2009] have observed that a complete migration may actually take tens of seconds, leading to a ten- to twentyfold increase in response time. In addition, we need to realize that during the migration, a service will be completely unavailable (i.e., unresponsive) for perhaps 4 seconds. The good news is that the response time goes up significantly only after the downtime to complete the migration, as shown in Figure 3.26.

In many cases, virtual machines are migrated to optimize the usage of actual machines. However, it may also be desirable to clone a virtual machine, for example, because the workload for the current machine is becoming too high. Such cloning is very similar to using multiple processes in concurrent servers by which a dispatcher process creates worker processes to handle incoming requests. This scheme was explained in Figure 3.4 when discussing multithreaded servers.

When cloning for this type of performance, it often makes more sense *not* to first copy memory pages, but, in fact, start with as few pages as possible as the service running on the cloned machine will essentially start anew. Note that this behavior is very similar to the usual parent-child behavior we see when forking a Unix process. Namely, the child will start with loading its own executable, thereby effectively cleaning the memory it inherited from its parent. This analogy inspired Lagar-Cavilla et al. [2009] to develop an analogous mechanism for *forking* a virtual machine. However, unlike the mechanism used traditionally for migrating virtual machines, their **VM fork** copies pages primarily on demand. The result is an extremely efficient cloning mechanism.

It is thus seen that there is no single best way to place copies of a virtual machine on different physical machines: it very much depends on how and why a virtual machine is being deployed.

3.6 Summary

Processes play a fundamental role in distributed systems as they form a basis for communication between different machines. An important issue is how processes are internally organized and, in particular, whether or not they support multiple threads of control. Threads in distributed systems are particularly useful to continue using the CPU when a blocking I/O operation is performed. In this way, it becomes possible to build highly-efficient servers that run multiple threads in parallel, of which several may be blocking to wait until disk I/O or network communication completes. In general, threads are preferred over the use of processes when performance is at stake.

Virtualization has since long been an important field in computer science, but in the advent of cloud computing has regained tremendous attention. Popular virtualization schemes allow users to run a suite of applications on top of their favorite operating system and configure complete virtual distributed systems in the cloud. Impressively enough, performance remains close to running applications on the host operating system, unless that system is shared with other virtual machines. The flexible application of virtual machines has led to different types of services for cloud computing, including infrastructures, platforms, and software — all running in virtual environments.

Organizing a distributed application in terms of clients and servers has proven to be useful. Client processes generally implement user interfaces, which may range from very simple displays to advanced interfaces that can handle compound documents. Client software is furthermore aimed at achieving distribution transparency by hiding details concerning the communication with servers, where those servers are currently located, and whether or not servers are replicated. In addition, client software is partly responsible for hiding failures and recovery from failures.

Servers are often more intricate than clients, but are nevertheless subject to only a relatively few design issues. For example, servers can either be iterative or concurrent, implement one or more services, and can be stateless or stateful. Other design issues deal with addressing services and mechanisms to interrupt a server after a service request has been issued and is possibly already being processed.

Special attention needs to be paid when organizing servers into a cluster. A common objective is to hide the internals of a cluster from the outside world. This means that the organization of the cluster should be shielded from applications. To this end, most clusters use a single entry point that can hand off messages to servers in the cluster. A challenging problem is to transparently replace this single entry point by a fully distributed solution.

Advanced object servers have been developed for hosting remote objects. An object server provides many services to basic objects, including facilities for storing objects, or to ensure serialization of incoming requests. Another important role is providing the illusion to the outside world that a collection

of data and procedures operating on that data correspond to the concept of an object. This role is implemented by means of object adapters. Object-based systems have come to a point where we can build entire frameworks that can be extended for supporting specific applications. Java has proven to provide a powerful means for setting up more generic services, exemplified by the highly popular Enterprise Java Beans concept and its implementation.

An exemplary server for Web-based systems is the one from Apache. Again, the Apache server can be seen as a general solution for handling a myriad of HTTP-based queries. By offering the right hooks, we essentially obtain a flexibly configurable Web server. Apache has served as an example not only for traditional Web sites, but also for setting up clusters of collaborative Web servers, even across wide-area networks.

An important topic for distributed systems is the migration of code between different machines. Two important reasons to support code migration are increasing performance and flexibility. When communication is expensive, we can sometimes reduce communication by shipping computations from the server to the client, and let the client do as much local processing as possible. Flexibility is increased if a client can dynamically download software needed to communicate with a specific server. The downloaded software can be specifically targeted to that server, without forcing the client to have it preinstalled.

Code migration brings along problems related to usage of local resources for which it is required that either resources are migrated as well, new bindings to local resources at the target machine are established, or for which systemwide network references are used. Another problem is that code migration requires that we take heterogeneity into account. Current practice indicates that the best solution to handle heterogeneity is to use virtual machines. These can take either the form of process virtual machines as in the case of, for example, Java, or through using virtual machine monitors that effectively allow the migration of a collection of processes along with their underlying operating system.